E.E.S.T. N° 5

Manual de instalación de MacroEntreno

Instalación, configuración y publicación de la app y la API

Alexis Vallejos
Materia: Proyecto, Diseño e Implementación de Sistemas Computacionales
Docente: Martín A. Estanga Schulze
Curso: 7° 4° GB
Fecha: 30 de octubre de 2026

Resumen

Este manual explica cómo instalar y poner en marcha MacroEntreno en una computadora y cómo publicarla en internet. Se detallan los programas necesarios, cómo descargar el código, cómo configurar y levantar la API con su base de datos MySQL, cómo ejecutar la app de React Native y cómo comprobar que todo funciona con las pruebas automáticas. También se muestra la publicación en Vercel con una base MySQL externa. Cada paso se ejecutó sobre una copia nueva del repositorio y las figuras muestran las salidas reales de la terminal.

Palabras clave: instalación, Node.js, MySQL, variables de entorno, Expo, Vercel.

Índice

Introducción **4**
Desarrollo **5**
Requisitos 5
Descargar el código 5
Instalar y configurar la API 5
Dependencias 5
Variables de entorno 6
Base de datos 6
Levantar la API 6
Pruebas automáticas de la API 7
Instalar y ejecutar la app 8
Dependencias y dirección de la API 8
Ejecutar 8
Pruebas y revisión del código de la app 9
Inicio de sesión con redes sociales 9
Generar el APK 10
Publicar en Vercel 10
Problemas frecuentes 12
Conclusión **13**

Introducción

MacroEntreno tiene dos partes que se instalan por separado. La **API** está hecha en Node.js con Express y guarda los datos en MySQL; se encarga del inicio de sesión, los permisos de cada rol y todos los datos. La **app** está hecha en React Native con Expo y funciona en Android, iPhone y en el navegador; muestra las pantallas y le pide los datos a la API.

Para comprobar que las instrucciones están completas, se siguieron desde cero sobre una copia recién descargada de GitHub, sin reutilizar nada de la carpeta de trabajo. Todo lo que aparece en las figuras es la salida obtenida en esa instalación.

Desarrollo

Requisitos

Tabla 1

Programas necesarios

Programa	Versión	Para qué se usa
Node.js y npm	20 o superior	Ejecutar la API y las herramientas de Expo
Git	Cualquiera reciente	Descargar el código del repositorio
MySQL	8.0 o superior	Base de datos (opcional en desarrollo: hay un modo con MySQL temporal)
Navegador o Expo Go	Expo Go para SDK 58	Abrir la app en la computadora o en el celular

Para ver qué versiones hay instaladas se usan los comandos de la Figura 1. En la instalación de prueba había Node.js 24, npm 11 y Git 2.39.

Figura 1

Versiones de Node.js, npm y Git

```
$ node -v
v24.21.0
$ npm -v
11.19.0
$ git --version
git version 2.39.5 (Apple Git-154)
```

Descargar el código

El código está en GitHub. Se clona con `git clone` y se entra a la carpeta (Figura 2). Adentro están `backend/` (la API), `mobile/` (la app), `api/` (la entrada de la API para Vercel), `docs/` (manuales y guías) y `vercel.json`.

Figura 2

Clonado del repositorio

```
$ git clone -b reestructuracion-react-native
https://github.com/AlexisVallejos/macroentreno_flet.git
Cloning into 'macroentreno_flet'...
$ cd macroentreno_flet && ls
README.md
api
backend
docs
mobile
vercel.json
```

Instalar y configurar la API

Dependencias

Dentro de `backend/` se instalan las dependencias con `npm ci`, que respeta las versiones exactas de `package-lock.json` (Figura 3).

Figura 3

Instalación de las dependencias de la API

```
$ cd backend && npm ci
added 160 packages in 4s
```

Variables de entorno

La configuración se guarda en el archivo `backend/.env`, que no se sube a Git porque tiene contraseñas. Se crea copiando el ejemplo con `cp .env.example .env` y se completan los datos (Figura 4). Las más importantes están en la Tabla 2.

Figura 4

Creación de backend/.env a partir del ejemplo

```
$ cp .env.example .env
$ grep -E "^(PORT|DB_|JWT_)" .env
PORT=3000
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=macroentreno
DB_SSL=false
DB_CONNECTION_LIMIT=10
JWT_ACCESS_SECRET=cambiar_por_un_secreto_largo_de_acceso
JWT_REFRESH_SECRET=cambiar_por_otro_secreto_largo_de_renovacion
JWT_ACCESS_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_DAYS=30
```

Tabla 2

Variables de entorno principales de la API

Variable	Ejemplo	Descripción
`PORT`	`3000`	Puerto donde escucha la API
`DB_HOST`, `DB_PORT`	`localhost`, `3306`	Servidor MySQL
`DB_USER`, `DB_PASSWORD`	`root`, …	Usuario y contraseña de MySQL
`DB_NAME`	`macroentreno`	Nombre de la base
`DB_SSL`	`false`	`true` si el servidor MySQL exige conexión cifrada
`JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`	96 caracteres	Secretos que firman los tokens; se generan con `openssl rand -hex 48`
`PUBLIC_API_URL`	`https://…`	Dirección pública de la API, para el login con redes sociales
`GOOGLE_CLIENT_ID`, `GITHUB_CLIENT_ID`, …	—	Credenciales de cada red social
`OPENAI_API_KEY`	`sk-…`	Opcional: activa el asistente con IA

Nota. La lista completa, con todas las variables y su explicación, está en `backend/.env.example` y en `docs/INSTALACION.md`.

Base de datos

Con MySQL instalado, `npm run db:init` crea la base, aplica las tablas de `database/schema.sql` y carga los datos de ejemplo: un usuario por rol, el catálogo de alimentos y ejercicios y una semana de historial. Los usuarios de ejemplo son `admin@macroentreno.com`, `coach@macroentreno.com`, `atleta@macroentreno.com` y `martina@macroentreno.com`, todos con la contraseña `Demo1234`. Este comando vacía las tablas, así que cuando la base no está en la misma computadora exige agregar `--yes` para confirmar.

Levantar la API

Con la base lista se ejecuta `npm run dev`. Para probar sin instalar MySQL existe `npm run dev:memory`, que descarga un MySQL temporal, carga los datos de ejemplo y levanta la API en un solo

paso; los datos se pierden al cerrarla. Es el modo que se usó en la instalación de prueba (Figura 5). Al arrancar, la API avisa si el asistente con IA está activo.

Figura 5

API levantada con un MySQL temporal

```
$ npm run dev:memory
> macroentreno-api@1.0.0 dev:memory
> node scripts/dev-memory.js
MySQL temporal en el puerto 11638. Usuarios demo con contraseña Demo1234.
MacroEntreno API escuchando en http://localhost:3000/api
Asistente con IA: DESACTIVADO (falta OPENAI_API_KEY) → pegá tu clave en backend/.env →
OPENAI_API_KEY y reiniciá la API
```

Para comprobar que la API funciona y llega a la base se consulta `/api/health` (Figura 6). La respuesta `"database":"ok"` confirma las dos cosas.

Figura 6

Comprobación del estado de la API

```
$ curl http://localhost:3000/api/health
{"data":{"status":"ok","database":"ok","time":"2026-10-30T18:58:45.453Z"}}
```

Pruebas automáticas de la API

Con `npm test` se ejecutan las pruebas del servidor. Cada archivo crea su propia base temporal, así que no hace falta tener MySQL instalado. En la instalación de prueba pasaron los 184 casos, en 25 grupos (Figura 7).

Figura 7

Resultado de las pruebas de la API

```
$ cd backend && npm test
✔ asistente con IA: estado de la configuración
✔ asistente con IA: errores de OpenAI
✔ asistente con IA: mensajes e historial
✔ salud y errores generales
✔ autenticación
✔ login social (flujo OAuth con proveedor simulado)
✔ roles y permisos
✔ alimentos
✔ diario
✔ ejercicios y entrenamientos
✔ progreso
✔ índices
✔ MacroEntreno Pro: micronutrientes
✔ MacroEntreno Pro: FatSecret
✔ MacroEntreno Pro: suscripción con Mercado Pago
✔ seguimiento, planes e informes por rol
✔ alimentos: código de barras y bases gratuitas
✔ asistente con IA
✔ registro: alta exitosa
✔ registro: validaciones por campo
✔ registro: email duplicado
✔ login: credenciales correctas
✔ login: credenciales incorrectas
✔ flujo completo: registro, login, refresh, logout y cambio de contraseña
✔ seguridad: intentos de saltear la validación
ℹ tests 184
ℹ suites 25
ℹ pass 184
ℹ fail 0
```

Instalar y ejecutar la app

Dependencias y dirección de la API

Dentro de `mobile/` se instalan las dependencias con `npm ci` (Figura 8). Después hay que indicarle a la app dónde está la API, en el archivo `mobile/.env`, con la variable `EXPO_PUBLIC_API_URL`. En el navegador de la misma computadora alcanza con `http://localhost:3000/api`. En un celular hay que usar la IP de la computadora en la red Wi-Fi, por ejemplo `http://192.168.0.10:3000/api`, porque para el teléfono "localhost" es él mismo. Si existe `mobile/.env.local`, tiene prioridad sobre `.env` y su puerto tiene que coincidir con el de la API.

Figura 8

Instalación de las dependencias de la app



Ejecutar

La app se inicia con `npx expo start`. Con `--web` se abre en el navegador (Figura 9); sin esa opción muestra un código QR que se escanea con Expo Go para abrirla en el celular, o se puede presionar `a` para un emulador de Android.

Figura 9

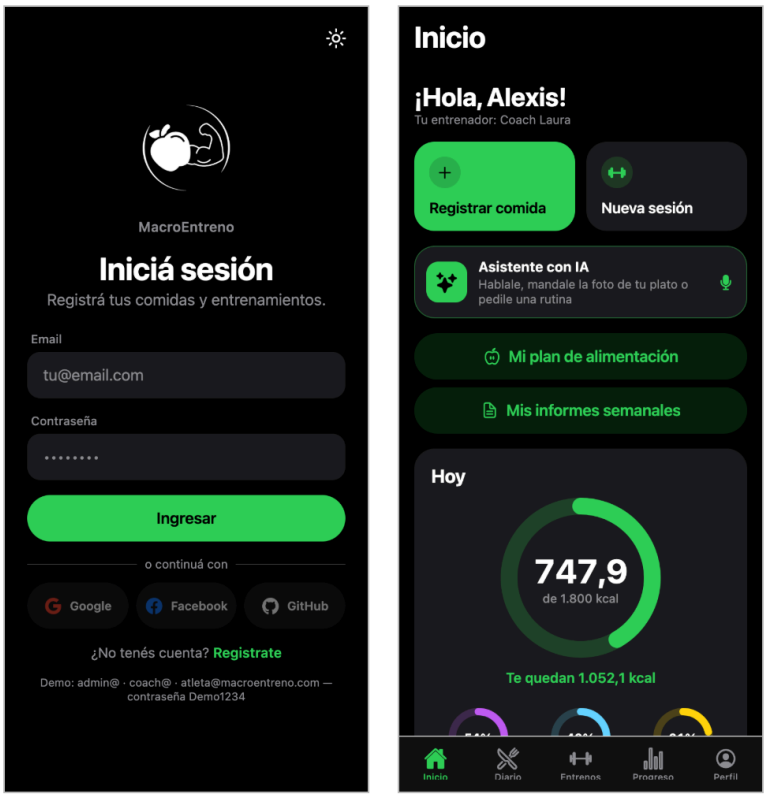
Inicio de la app con Expo

```
$ cd mobile && npx expo start --web
env: load .env
env: export EXPO_PUBLIC_API_URL
Starting project at ~/macroentreno_flet/mobile
Using src/app as the root directory for Expo Router.
Metro is running in CI mode, reloads are disabled. Remove CI=true to enable watch mode.
Starting Metro Bundler
Waiting on http://localhost:8081
Logs for your project will appear below.
Web Bundled 2108ms node_modules/expo-router/entry.js (1121 modules)
```

En la Figura 10 se ve la app funcionando en la instalación de prueba: la pantalla de inicio de sesión y, después de ingresar con la cuenta de ejemplo del Atleta, su resumen del día con los datos que devolvió la API.

Figura 10

App funcionando contra la API local



(a) Inicio de sesión (b) Inicio del Atleta

Pruebas y revisión del código de la app

La app tiene sus propias pruebas de las validaciones de los formularios (`npm test`) y un revisor de código (`npx expo lint`). Las 53 pruebas pasaron y el revisor no informó errores ni advertencias (Figura 11).

Figura 11

Pruebas y revisión del código de la app

```
$ cd mobile && npm test
ℹ tests 53
ℹ suites 5
ℹ pass 53
ℹ fail 0
$ npx expo lint
(sin errores ni advertencias)
```

Inicio de sesión con redes sociales

El ingreso con Google, Facebook y GitHub lo resuelve la API con OAuth 2.0. Para cada red se crea una aplicación en su consola de desarrolladores, se registra como dirección de retorno `<PUBLIC_API_URL>/api/auth/oauth/<red>/callback` y se copian el identificador y el secreto en `backend/.env` (Tabla 3). Google y Facebook exigen HTTPS, por eso en desarrollo hace falta un túnel como ngrok o la versión publicada en Vercel. Mientras una red no está configurada, la app muestra su botón atenuado y avisa que todavía no está disponible.

Tabla 3

Configuración de cada red social

Red	Dónde se crea	Variables
Google	console.cloud.google.com → Google Auth Platform → Clientes	`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`
Facebook	developers.facebook.com → Mis apps	`FACEBOOK_APP_ID`, `FACEBOOK_APP_SECRET`
GitHub	github.com → Settings → Developer settings → OAuth Apps	`GITHUB_CLIENT_ID`, `GITHUB_CLIENT_SECRET`

Generar el APK

Para instalar la app en un Android sin Expo Go se genera un APK con EAS Build, el servicio de compilación de Expo. Primero se escribe en `mobile/eas.json`, perfil `preview`, la dirección de la API que va a quedar dentro del APK. Después se ejecuta `npx eas-cli@latest build -p android --profile preview`. Al terminar, EAS muestra un link para descargar el archivo `.apk` e instalarlo en el teléfono.

Publicar en Vercel

La versión web y la API se publican juntas en Vercel, en **https://macroentreno.vercel.app**. El archivo `vercel.json` indica cómo armar el proyecto: instala la API y la app, exporta la app para web con la API en `/api` y envía todos los pedidos a `/api` a la función de `api/index.mjs`, que usa la misma API de Express. Vercel no aloja MySQL, por eso la base está en un servidor propio administrado con Easy-Panel, con el puerto expuesto a internet.

Tabla 4

Pasos para publicar en Vercel

#	Paso	Comando
1	Cargar las tablas y los datos en la base remota	`DB_HOST=… DB_PORT=… npm run db:init -- -- yes`
2	Iniciar sesión en Vercel y vincular el proyecto	`npx vercel login` · `npx vercel link --project macroentreno`
3	Cargar las variables de entorno: secretos JWT, datos de la base, `DB_CONNECTION_LIMIT=3` y `CRON_SECRET`	`printf %s "valor" | npx vercel env add NOMBRE production`
4	Publicar	`npx vercel deploy --prod`
5	Comprobar la API y la base	`curl https://macroentreno.vercel.app/api/health`

En la Figura 12 se ve la comprobación de la versión publicada: `/api/health` responde que la base está disponible y el inicio de sesión con la cuenta de ejemplo devuelve los datos del usuario. La Figura 13 muestra la app abierta desde la dirección de Vercel con esa cuenta.

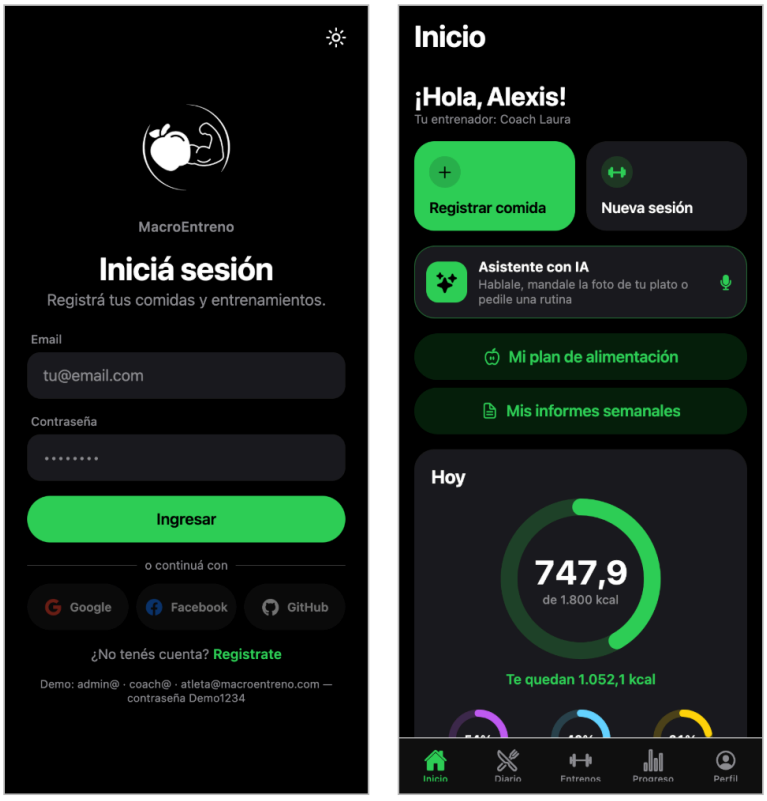
Figura 12

Comprobación de la versión publicada en Vercel

```
$ curl https://macroentreno.vercel.app/api/health
{"data":{"status":"ok","database":"ok","time":"2026-10-30T19:01:47.413Z"}}
$ curl -X POST https://macroentreno.vercel.app/api/auth/login -d
{"email":"atleta@…","password":"…"}
{"data":{"user":{"id":3,"name":"Alexis
Atleta","email":"atleta@macroentreno.com","role":"a…
```

Figura 13

App publicada en macroentreno.vercel.app



(a) Inicio de sesión (b) Inicio del Atleta

En Vercel no hay un servidor encendido todo el tiempo, por eso las tareas que la API hace periódicamente (generar los informes semanales y borrar sesiones vencidas) las dispara un cron de Vercel una vez por día, en `/api/cron/diario`.

Problemas frecuentes

Tabla 5

Problemas frecuentes y cómo resolverlos

Síntoma	Causa probable	Solución
"No se pudo conectar con el servidor"	La app no llega a la API	Revisar `EXPO_PUBLIC_API_URL` (IP y puerto), que la API esté corriendo y que el celular esté en la misma Wi-Fi
`/api/health` responde 503	La API no llega a MySQL	Revisar `DB_HOST`, `DB_PORT`, usuario y contraseña, y que el puerto de MySQL esté expuesto
La app usa otro puerto	`mobile/.env.local` pisa a `.env`	Igualar el puerto de `.env.local` con el `PORT` de la API
`db:init` pide `--yes`	La base no es local	Confirmar agregando `--yes` si de verdad se quiere vaciar y recargar
Botón de Google atenuado	La red no está configurada	Cargar sus credenciales (Tabla 3) y reiniciar la API
Google: `redirect_uri_mismatch`	La dirección de retorno no coincide	Registrar exactamente `<PUBLIC_API_URL>/api/auth/oauth/google/callback`
Expo Go: "incompatible SDK"	Expo Go desactualizado	Actualizar Expo Go: el proyecto usa SDK 58
"Tu sesión expiró"	Se cerraron las sesiones (cambio de contraseña o baja)	Volver a iniciar sesión

Conclusión

Siguiendo estos pasos sobre una copia nueva del repositorio, MacroEntreno quedó funcionando en una computadora: la API levantó con su base de datos, respondió a la comprobación de estado y pasó sus 184 pruebas, y la app abrió en el navegador e inició sesión con la cuenta de ejemplo. La misma configuración permitió publicarla en Vercel con una base MySQL externa, donde también se comprobó que la API llega a la base y que se puede ingresar.

La instalación mostró la importancia de las variables de entorno: la mayoría de los problemas que pueden aparecer, como una app que no encuentra la API o una API que no llega a la base, se resuelven revisando una dirección, un puerto o una contraseña. Mantener esos datos fuera del código, en archivos `.env` que no se suben a Git, permite usar el mismo proyecto en la computadora, en el celular y en internet sin cambiar una sola línea.